

Atividade 12 - Tabelas Hash com Endereçamento Aberto

28 de novembro de 2025

Problema A

```
1 #include <stdlib.h>
2
3 typedef struct TabelaHash {
4     int *tabela;
5     int tamanho;
6 } TabelaHash;
7
8 int funcaoHash(int chave, int tamanho) {
9     return abs(chave) % tamanho;
10 }
11
12 TabelaHash *criaTabelaHash(int tamanho){
13     TabelaHash *th = (TabelaHash*) malloc (sizeof(TabelaHash));
14
15     th->tabela = (int*) calloc (tamanho, sizeof(int));
16     th->tamanho = tamanho;
17     return th;
18 }
19
20 void insereTabelaHash(TabelaHash *th, int chave){
21     if(th == NULL) return;
22
23     int pos = funcaoHash(chave, th->tamanho);
24
25     for(int i = 0; i < th->tamanho; ++i){
26         int j = (pos + i) % th->tamanho;
27
28         if(th->tabela[j] == 0){
29             th->tabela[j] = chave;
30             return;
31         }
32     }
33 }
34
35 void imprimeTabelaHash(TabelaHash *th){
36     for(int i = 0; i < th->tamanho; ++i){
37         printf("%d %d\n", i, th->tabela[i]);
38     }
39     printf("\n");
40 }
```

```
41
42 void liberaTabelaHash(TabelaHash *th){
43     free(th->tabela);
44     free(th);
45 }
46
47 int main(){
48     int n;
49     scanf("%d", &n);
50     for(int i = 0; i < n; ++i){
51         int tamanho, c;
52         scanf("%d %d", &tamanho, &c);
53         TabelaHash *th = criaTabelaHash(tamanho);
54
55         for(int j = 0; j < c; ++j){
56             int chave;
57             scanf("%d", &chave);
58             insereTabelaHash(th, chave);
59         }
60
61         imprimeTabelaHash(th);
62         liberaTabelaHash(th);
63     }
64     return 0;
65 }
```

Problema B

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct TabelaHash {
5     int *tabela;
6     int tamanho;
7 } TabelaHash;
8
9 int funcaoHash(int chave, int tamanho) {
10     return abs(chave) % tamanho;
11 }
12
13 TabelaHash *criaTabelaHash(int tamanho){
14     TabelaHash *th = (TabelaHash*) malloc (sizeof(TabelaHash));
15     th->tabela = (int*) calloc (tamanho, sizeof(int));
16     th->tamanho = tamanho;
17
18     return th;
19 }
20
21 void insereTabelaHash(TabelaHash *th, int chave){
22     if(th == NULL) return;
23
24     int pos = funcaoHash(chave, th->tamanho);
25
26     for(int i = 0; i < th->tamanho; ++i){
27         int j = (pos + i) % th->tamanho;
28
29         if(th->tabela[j] == 0 || th->tabela[j] == -1){
30             th->tabela[j] = chave;
31             return;
32         }
33     }
34 }
35
36 int buscaTabelaHash(TabelaHash *th, int chave){
37     if(th == NULL || th->tabela == NULL) return -1;
38
39     int pos = funcaoHash(chave, th->tamanho);
40
41     for(int i = 0; i < th->tamanho; ++i){
42         int j = (pos + i) % th->tamanho;
43
44         if(th->tabela[j] == chave) return j;
45         if(th->tabela[j] == 0) return -1;
46     }
47     return -1;
48 }
49
50 void removeTabelaHash(TabelaHash *th, int chave){
51     if(th == NULL || th->tabela == NULL) return;
52
53     int pos = funcaoHash(chave, th->tamanho);
54
55     for(int i = 0; i < th->tamanho; ++i){
56         int j = (pos + i) % th->tamanho;
```

```
57
58     if(th->tabela[j] == chave){
59         th->tabela[j]= -1;
60         return;
61     }
62 }
63 }
64
65 void liberaTabelaHash(TabelaHash *th){
66     if(th == NULL) return;
67
68     if(th->tabela != NULL)
69         free(th->tabela);
70
71     free(th);
72 }
73
74 int main(){
75     int n;
76     scanf("%d\n", &n);
77     for(int i = 0; i < n; ++i){
78         int tamanho, c;
79         scanf("%d %d\n", &tamanho, &c);
80         TabelaHash *th = criaTabelaHash(tamanho);
81
82         for(int j = 0; j < c; ++j){
83             char op;
84             int chave;
85             scanf("%c %d\n", &op, &chave);
86
87             if(op == 'I'){
88                 insereTabelaHash(th, chave);
89             } else if (op == 'B'){
90                 printf("%d\n", buscaTabelaHash(th, chave));
91             } else if (op == 'R'){
92                 removeTabelaHash(th, chave);
93             }
94         }
95         printf("\n");
96
97         liberaTabelaHash(th);
98     }
99     return 0;
100 }
```